Machine Learning Operations (MLOps)
AIMLCZG523

Assignment 01
Heart Disease Prediction Pipeline

BITS Pilani WILP

Student Name: PIYUSH KUMAR

Student ID: 2025CT05044

Repository: 2025ct05044-cmd: MLOps-Assignment

20 October 2026

1. Executive Summary and Objectives

This assignment helped me understand how a machine-learning model can be developed and then prepared for use as an application. I worked through the main stages of an MLOps workflow: preparing the heart-disease data, exploring it with visualizations, processing features, comparing classification models, tracking experiments with MLflow, testing the API, building a Docker image, and deploying the service to a local Kubernetes cluster. The aim was not only to train a model, but also to make the workflow easier to reproduce and verify.

For inference, I created a FastAPI service that loads the saved scikit-learn pipeline and accepts prediction input as JSON. I built a Docker image to package the application and its dependencies. I then loaded that image into Minikube and applied the Kubernetes Deployment and Service manifests. The pod reached the Running state with one of one containers ready. Because the Minikube tunnel could not start without an SSH executable on the Windows PATH, I verified the service through kubectl port-forward instead. The health endpoint returned status ok and model_loaded true. This confirms local deployment, not public internet deployment.

The model is being used only as an academic demonstration of machine-learning and MLOps techniques. Its predictions should not be used to diagnose, treat, or make healthcare decisions. A real clinical system would require appropriate clinical validation, representative data, privacy and security controls, and professional oversight.

- Build a reproducible classification pipeline.
- Compare candidate models using cross-validation and held-out test metrics.
- Track runs, parameters, metrics, plots, and model artifacts with MLflow.
- Package inference behind a JSON API and validate it with tests and CI.
- Deploy locally to Kubernetes and record evidence.

2. Notebook Structure and Workflow

The project is organized into four Jupyter notebooks. Each notebook focuses on a specific stage of the machine learning and MLOps lifecycle, making the workflow easier to understand, execute, and maintain.

1. 01_EDA_and_Data_Preparation.ipynb — Exploratory Data Analysis and Data Preparation

This notebook focuses on understanding and preparing the heart disease dataset. It covers data inspection, dataset dimensions, data types, missing-value checks, target distribution, descriptive statistics, and exploratory visualizations. These steps help identify data characteristics and prepare the dataset for subsequent model development.

2. 02_Model_Development_v1.ipynb — Model Development

This notebook focuses on developing and evaluating machine learning models for heart disease classification. It provides the model-development stage of the workflow, where prepared data is used to train classification models and assess their performance using appropriate evaluation metrics. The resulting evaluation outputs support model comparison and selection.

3. 03_MLflow_Experiment_Tracking.ipynb — Experiment Tracking

This notebook integrates MLflow into the model-development workflow. It records experiment information, including relevant parameters, evaluation metrics, and supporting artifacts. The recorded runs and visualizations help compare experiments and maintain a history of model-development results.

4. 04_FastAPI_Model_Serving.ipynb — Model Serving

This notebook focuses on serving the trained machine learning pipeline through an API. It demonstrates the model-serving stage, connecting the saved model with an application interface so that input features can be submitted for prediction. This stage links model development with application deployment and testing.

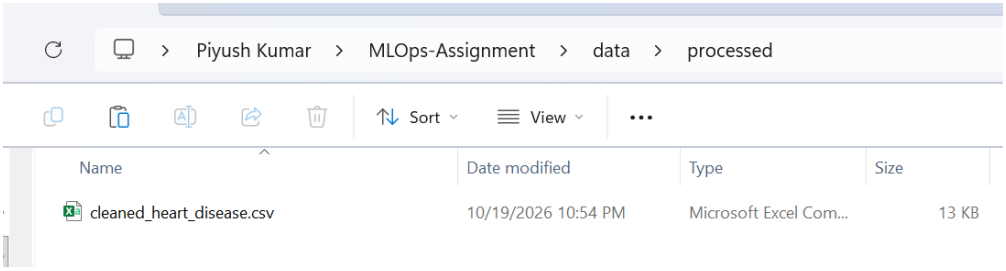
Together, these four notebooks represent the progression from data understanding and preparation to model development, experiment tracking, and model serving. They provide a notebook-based view of the project, while the source code, saved model, tests, Docker configuration, and Kubernetes manifests support the application and deployment stages.

3. Dataset and Data Preparation

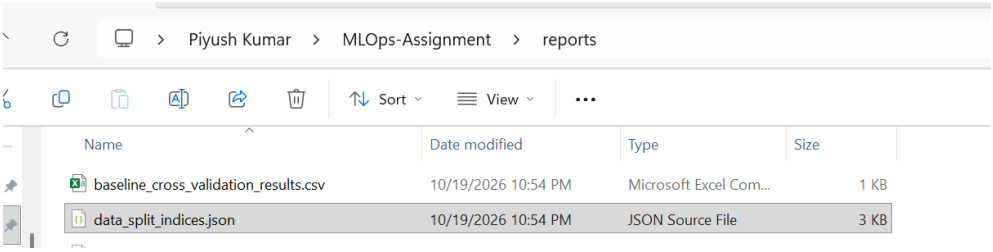
The processed dataset used in this project is saved at data/processed/cleaned_heart_disease.csv. The recorded dataset contains 297 rows and 14 columns, including the target column. The target distribution contains 160 records in class 0 and 137 records in class 1. This is a fairly balanced distribution, although the two classes are not exactly equal. The preprocessing notes report no missing values in the processed file. I used these checks to understand the data shape and target distribution before model development.

The split indices are stored in reports/data_split_indices.json. Saving these indices provides a record of which observations were assigned to the evaluation split and helps make comparisons more consistent. The fitted preprocessing pipeline is also important: the same transformations must be applied when the API receives new records. Keeping preprocessing and prediction together reduces the risk of training the model on one representation of the data and serving it with a different representation.

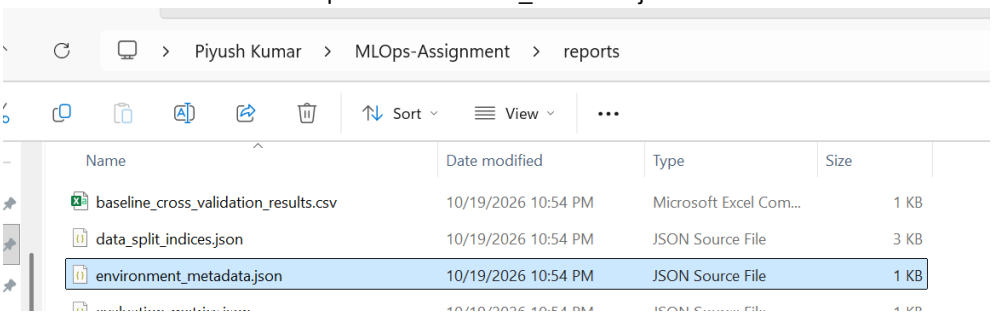
- Processed data: data/processed/cleaned_heart_disease.csv



- Split metadata: reports/data_split_indices.json



- Environment metadata: reports/environment_metadata.json



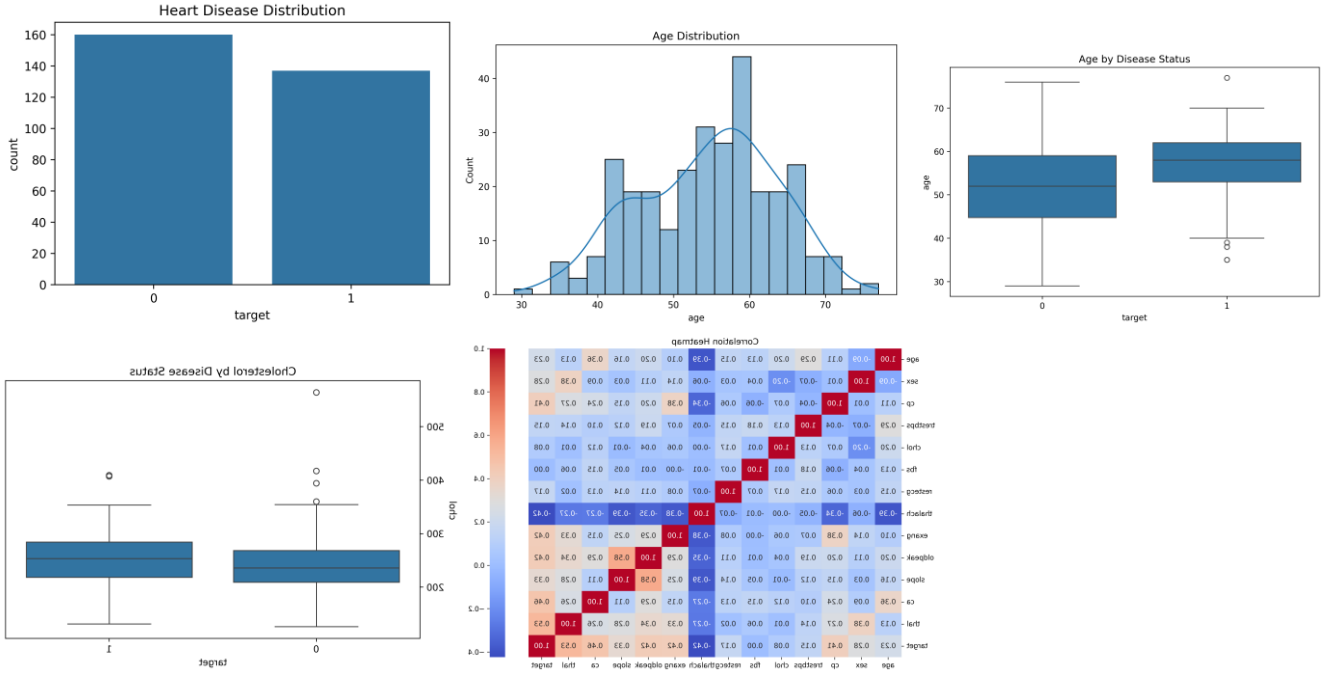
4. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was used to understand the data before fitting models. The class-balance chart shows how many observations belong to each target class. The age distribution helps show which age groups are represented in the dataset. The age-versus-disease and cholesterol boxplots make it easier to compare the distributions across target classes and to identify possible outliers. The correlation heatmap summarizes pairwise linear associations between numeric variables. I treated these charts as tools for understanding the dataset, not as proof that one feature causes heart disease.

The class counts indicate modest imbalance rather than an extreme imbalance, so both classes are represented in the processed data. A correlation does not establish causation, and a strong relationship in a small dataset should be interpreted carefully. Boxplots may show unusual values, but an outlier should not be removed only because it looks different; removal would need a data-quality or domain-based justification. The EDA figures retained in this report provide visual evidence for these observations.

- screenshots/class_balance.png
- screenshots/age_distribution.png

- screenshots/age_disease_boxplot.png
- screenshots/cholesterol_boxplot.png
- screenshots/correlation_heatmap.png



5. Feature Engineering and Pipeline

The trained pipeline is saved as models/heart_disease_pipeline.joblib. A pipeline is useful because it keeps the required preprocessing steps and the estimator together in one artifact. During inference, the API loads this saved object and uses it to produce predictions, rather than independently rebuilding preprocessing logic. This supports consistency between training and serving and makes packaging simpler.This project pins scikit-learn to version 1.6.1.

Feature scaling is part of the project workflow because features can be measured on different numeric ranges. Scaling helps algorithms such as Logistic Regression learn from features without one large-scale feature dominating only because of its units. The target is treated as a binary classification label, with class 0 and class 1 representing the dataset's two categories.

Pipeline: models/heart_disease_pipeline.joblib

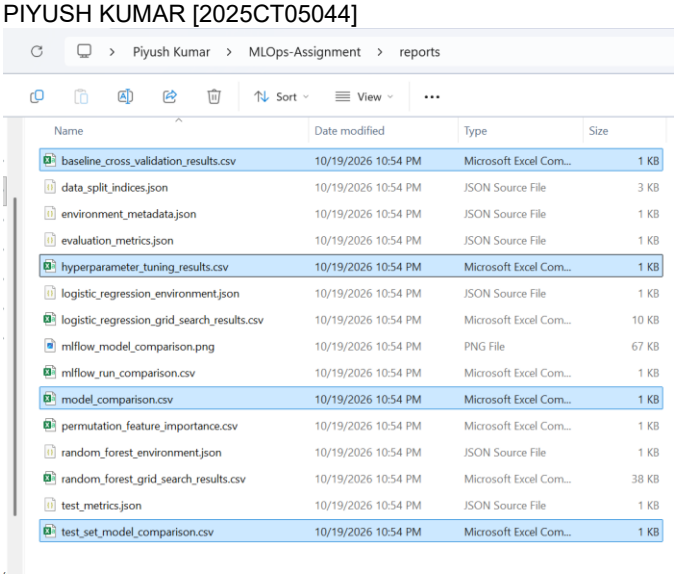
- Feature engineering: src/feature_engineering.py
- Preprocessing: src/data_preprocessing.py
- Dependencies: requirements.txt

6. Model Development and Evaluation

I compared Logistic Regression and Random Forest using the project's saved comparison tables. Logistic Regression provides a relatively simple baseline, while Random Forest combines decision trees and can represent nonlinear patterns. Cross-validation results help assess performance across multiple folds, and grid-search outputs record the tested hyperparameter combinations. Held-out test results provide a separate estimate of performance on data not used to fit the selected model. The results should always be reported with the evaluation scope and model configuration clearly identified.

Files include:

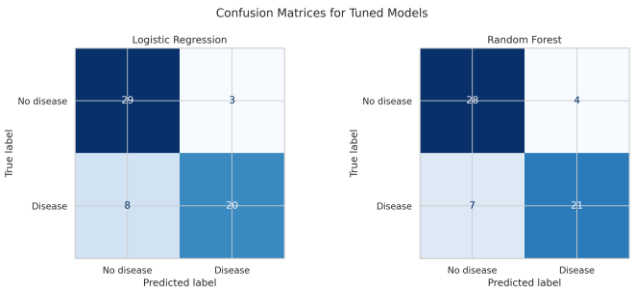
reports/model_comparison.csv
reports/test_set_model_comparison.csv
reports/baseline_cross_validation_results.csv
reports/hyperparameter_tuning_results.csv.



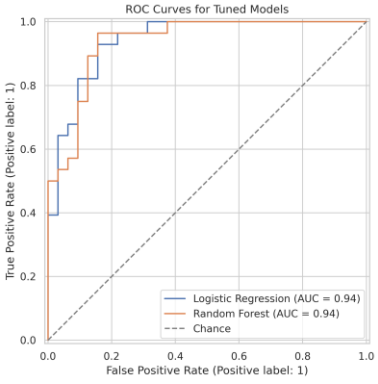
The local MLflow database recorded eight runs for the experiment heart-disease-model-development: six with status FINISHED and two with status FAILED. In the inspected finished-run output, the highest recorded test ROC-AUC was approximately 0.9408, with test accuracy approximately 0.8167 and mean cross-validation ROC-AUC approximately 0.9028 for the associated experiment. These values describe a recorded run and should not be treated as a universal score for the project. Some rows repeat metric values, so the total number of runs should not be described as eight distinct model configurations. The failed runs are also useful evidence because experiment tracking records unsuccessful attempts instead of hiding them.

Evaluation reports:

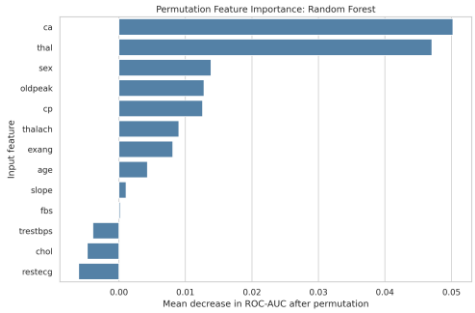
reports/evaluation_metrics.json
reports/test_metrics.json
Confusion matrices:



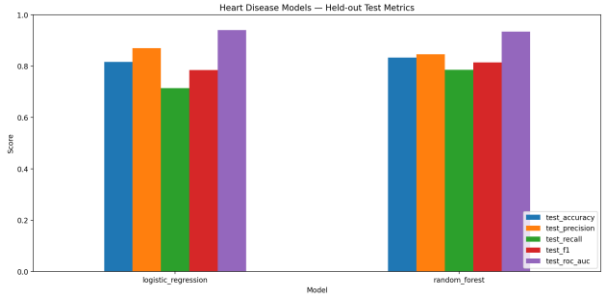
screenshots/model_confusion_matrices.png



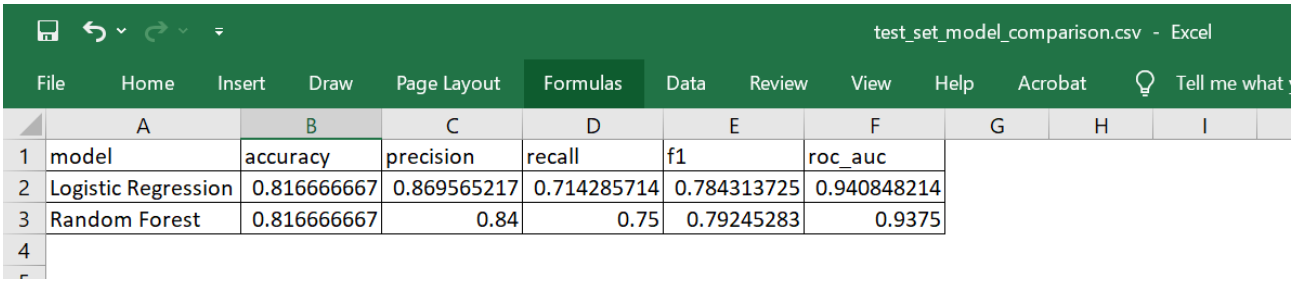
- ROC curves: screenshots/model_roc_curves.png
- Feature importance:



- screenshots/permutation_feature_importance.png



- Comparison plot: reports/mlflow_model_comparison.png
- Metric table from reports/test_set_model_comparison.csv

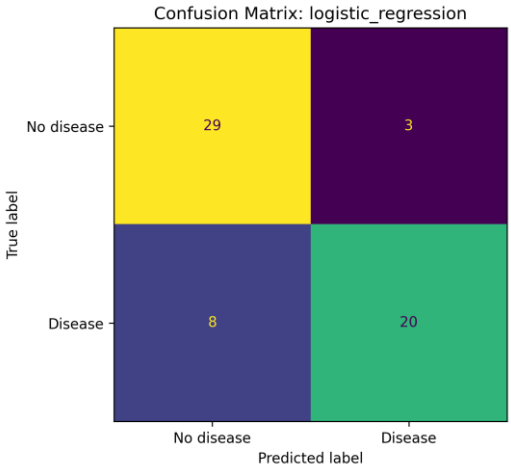


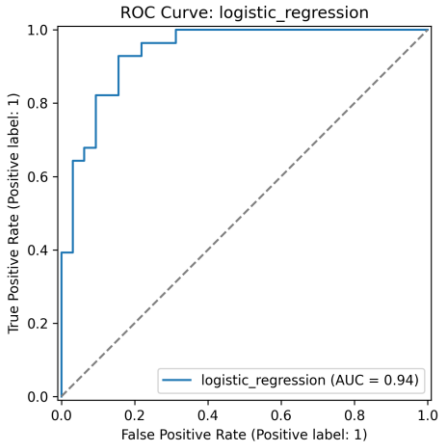
	A	B	C	D	E	F	G	H	I
1	model	accuracy	precision	recall	f1	roc_auc			
2	Logistic Regression	0.816666667	0.869565217	0.714285714	0.784313725	0.940848214			
3	Random Forest	0.816666667	0.84	0.75	0.79245283	0.9375			
4									

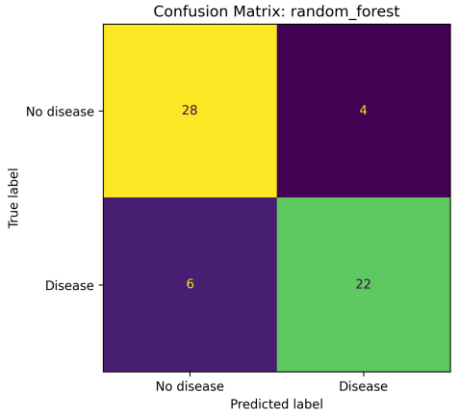
7. MLflow Experiment Tracking

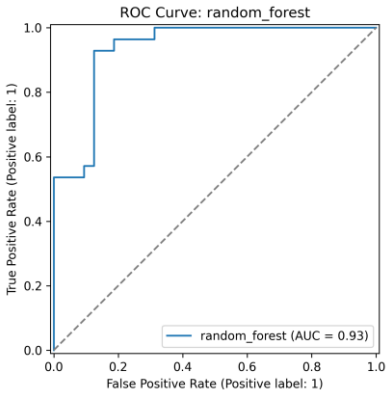
I used MLflow 3.17.1 to organize and inspect model-development experiments. The tracking database is mlflow.db, and the experiment is named heart-disease-model-development. A query of the local database returned eight runs, of which six were FINISHED and two were FAILED. MLflow helps keep run-level metrics and artifacts associated with an experiment, which is more reliable than recording every result manually in notes.

The mlruns directory stores run artifacts, including environment metadata and model-specific evaluation plots. Confusion matrices show the counts of correct and incorrect predictions for each class. ROC curves show the trade-off between true-positive and false-positive rates across thresholds, while ROC-AUC summarizes discrimination across those thresholds. I used these artifacts alongside the CSV reports rather than relying on a single metric.



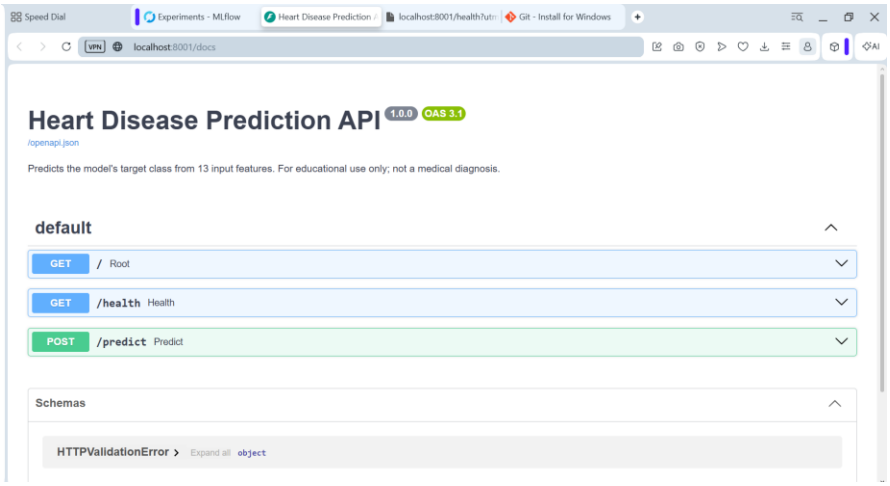


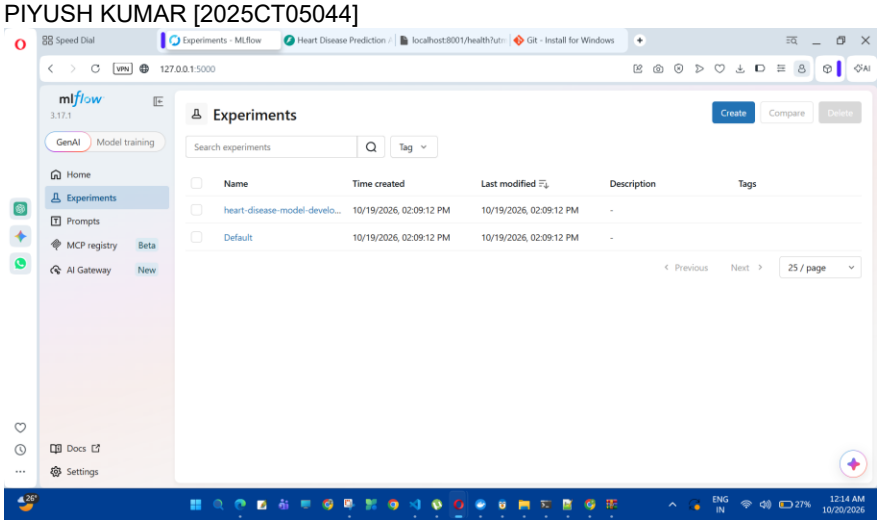




The notebook Notebook/03_MLflow_Experiment_Tracking.ipynb contains the experiment-tracking workflow. It logs run information such as parameters, cross-validation metrics, test metrics, plots, and model-related artifacts. This notebook is the main place to inspect how tracking is performed. The local MLflow UI was available at http://127.0.0.1:5000 while the UI process was running; this address is only accessible from the local machine and is not a public service URL.

The local UI was available at http://127.0.0.1:5000; this is not a public URL.





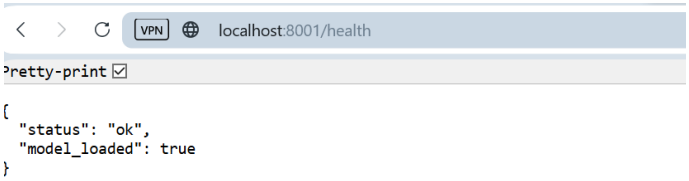
- Database: mlflow.db
- Artifacts: mlruns/
- Notebook: Notebook/03_MLflow_Experiment_Tracking.ipynb

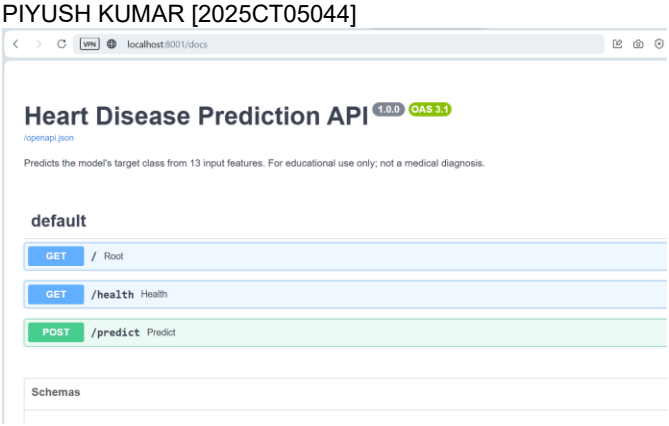
8. Packaging, API, and Automated Testing

The inference service is implemented in api/app.py using FastAPI. The root endpoint (GET /) provides basic service information, GET /health reports whether the service is running and whether the model has loaded, and POST /predict accepts feature values as JSON. The prediction response includes the predicted class, the estimated probability for class 1, the model identifier, and an educational-use disclaimer. Swagger documentation makes it possible to inspect the request schema and test the endpoints interactively without writing a separate client program.

The Dockerfile packages the API code, dependency installation, and saved model into an image that listens on port 8000. Pinning compatible dependencies is important because serialized scikit-learn objects can fail to load when the runtime library version differs from the version used to create the artifact. The automated test suite previously reported seven passing tests, and an earlier GitHub Actions run completed its test-and-train and Docker-build jobs successfully.

- API: api/app.py
- Container: Dockerfile
- Tests: tests/
- Workflow: .github/workflows/ml_pipeline.yml
- Local docs during port forwarding: http://localhost:8001/docs





Docker Build snap:

```
PS C:\Users\91620\MLOps-Assignment> docker build --no-cache -t heart-disease-api:latest .
[+] Building 54.4s (11/11) FINISHED                                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                              0.0s
 => => transferring dockerfile: 538B                                                                                              0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                                               2.3s
 => [internal] load .dockerignore                                                                                                 0.0s
 => => transferring context: 449B                                                                                                 0.0s
 => [1/6] FROM docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9ea65f6ce         0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9ea65f6ce         0.0s
 => [internal] load build context                                                                                                 0.0s
 => => transferring context: 378B                                                                                                 0.0s
 => CACHED [2/6] WORKDIR /app                                                                                                     0.0s
 => [3/6] COPY requirements.txt .                                                                                                 0.0s
 => [4/6] RUN python -m pip install --no-cache-dir --upgrade pip     && python -m pip install --no-cache-dir -r requirements.    34.6s
 => [5/6] COPY api/ ./api/                                                                                                        0.1s
 => [6/6] COPY models/heart_disease_pipeline.joblib ./models/heart_disease_pipeline.joblib                                        0.0s
 => exporting to image                                                                                                           17.1s
 => => exporting layers                                                                                                          13.4s
 => => exporting manifest sha256:e13d9cab1076531e9226b4261d83fc7199ac4877a53e8eed062da7da306f1346                                  0.0s
 => => exporting config sha256:16405fc15264cca3ea5f4c93a216fdc0c2ae70ad3131d94fecb7a22573b8f360                                    0.0s
 => => exporting attestation manifest sha256:5a5e6addc5325f1865c53b1b8fb3de51ad019107113d2961886214aa7b29f4ab                      0.0s
 => => exporting manifest list sha256:50a26bdc589e0267747bd15d4f8bca7f2eb9d62463a133076c1379e23b96785d                             0.0s
 => => naming to docker.io/library/heart-disease-api:latest                                                                       0.0s
 => => unpacking to docker.io/library/heart-disease-api:latest                                                                    3.6s
```

```
PS C:\Users\91620\MLOps-Assignment> docker images heart-disease-api
                                                                                                          i Info →   U  In Use
IMAGE                       ID             DISK USAGE   CONTENT SIZE   EXTRA
heart-disease-api:latest    9120cbe986a9        795MB          182MB
PS C:\Users\91620\MLOps-Assignment> docker run -d --name heart-disease-container -p 8000:8000 heart-disease-api:latest
56d346b8ad3367a3d00ded9c20ba384a98841c5d7bf27f7ab598a8674d02b77b
```

```
PS C:\Users\91620\MLOps-Assignment> docker run -d --name heart-disease-container -p 8000:8000 heart-disease-api:latest
20c5792ec3be4e443366a734f61433ca8e84d886d7cd5b9629bbaee296d78f18
PS C:\Users\91620\MLOps-Assignment> docker ps
CONTAINER ID   IMAGE                       COMMAND                  CREATED         STATUS         PORTS                                       NAMES
20c5792ec3be   heart-disease-api:latest    "uvicorn api.app:app…"   9 seconds ago   Up 8 seconds   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   heart-
disease-container
PS C:\Users\91620\MLOps-Assignment> docker ps
CONTAINER ID   IMAGE                       COMMAND                  CREATED          STATUS          PORTS                                       NAMES
20c5792ec3be   heart-disease-api:latest    "uvicorn api.app:app…"   13 seconds ago   Up 12 seconds   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   heart-
disease-container
PS C:\Users\91620\MLOps-Assignment> Invoke-RestMethod http://localhost:8000/health

status model_loaded
------ ------------
ok             True
```

9. Kubernetes Deployment

I deployed the container to a local Minikube cluster using the Docker driver. The Kubernetes Deployment is named heart-disease-api and requests one replica. The Service manifest uses type LoadBalancer, but in this local environment that does not automatically mean the application has a public IP address. The image was loaded into Minikube and the Kubernetes manifests were applied from the deployment directory.

The pod was verified as READY 1/1 and STATUS Running, with zero restarts at the time of inspection. I accessed the API using kubectl port-forward because the Minikube tunnel attempt failed with an error indicating that the ssh executable was not available on PATH. The command used was kubectl port-forward service/heart-disease-api 8001:8000. While that command

was running, the health endpoint returned status ok and model_loaded true, and the Swagger interface was available at http://localhost:8001/docs. This is local access only; a public endpoint remains outside the completed work.

Command used: kubectl port-forward service/heart-disease-api 8001:8000. The health endpoint returned status ok and model_loaded true. Swagger UI was available at http://localhost:8001/docs while forwarding was active. These are local-only URLs.

- Manifests: deployment/deployment.yaml and deployment/service.yaml
- Kubernetes evidence includes the pod and Service status, the health response, and local API access.
- The application was verified locally; a public cloud endpoint is not part of this deployment.

```
PS C:\Users\91620\MLOps-Assignment> kubectl version --client
Client Version: v1.36.1
Kustomize Version: v5.8.1
PS C:\Users\91620\MLOps-Assignment> & "C:\Program Files\Kubernetes\Minikube\minikube.exe" start --driver=docker
* minikube v1.39.0 on Microsoft Windows 11 Home Single Language 25H2
* Using the docker driver based on user configuration
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.51 ...
    > gcr.io/k8s-minikube/kicbase:  16.30 MiB / 507.61 MiB  3.21% 1.41 MiB p/s * Downloading Kubernetes v1.37.0 preload ...
    > gcr.io/k8s-minikube/kicbase:  507.61 MiB / 507.61 MiB  100.00% 8.65 MiB p
    > preloaded-images-k8s-v18-v1...:  347.58 MiB / 347.58 MiB  100.00% 6.55 Mi
* Creating docker container (CPUs=2, Memory=3072MB) ...
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Configuring CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Users\91620\MLOps-Assignment> kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   93s   v1.37.0
PS C:\Users\91620\MLOps-Assignment> docker stop heart-disease-container
heart-disease-container
```

```
PS C:\Users\91620\MLOps-Assignment> New-Item -ItemType Directory -Force deployment


    Directory: C:\Users\91620\MLOps-Assignment


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/19/2026  10:54 PM                deployment


PS C:\Users\91620\MLOps-Assignment> @'
>> apiVersion: apps/v1
>> kind: Deployment
>> metadata:
>>   name: heart-disease-api
>> spec:
>>   replicas: 1
>>   selector:
>>     matchLabels:
>>       app: heart-disease-api
>>   template:
>>     metadata:
>>       labels:
>>         app: heart-disease-api
>>     spec:
>>       containers:
>>         - name: heart-disease-api
>>           image: heart-disease-api:latest
>>           imagePullPolicy: Never
>>           ports:
>>             - containerPort: 8000
>> '@ | Set-Content -Encoding utf8 deployment\deployment.yaml
```

```
PS C:\Users\91620\MLOps-Assignment> @'
>> apiVersion: v1
>> kind: Service
>> metadata:
>>   name: heart-disease-api
>> spec:
>>   type: LoadBalancer
>>   selector:
>>     app: heart-disease-api
>>   ports:
>>     - port: 8000
>>       targetPort: 8000
>> '@ | Set-Content -Encoding utf8 deployment\service.yaml
PS C:\Users\91620\MLOps-Assignment> & "C:\Program Files\Kubernetes\Minikube\minikube.exe" image load heart-disease-api:latest
PS C:\Users\91620\MLOps-Assignment> kubectl apply -f deployment\
deployment.apps/heart-disease-api created
service/heart-disease-api created
PS C:\Users\91620\MLOps-Assignment> kubectl get pods
NAME                                 READY   STATUS    RESTARTS   AGE
heart-disease-api-6db5cf645c-2wmtx   1/1     Running   0          23s
PS C:\Users\91620\MLOps-Assignment> & "C:\Program Files\Kubernetes\Minikube\minikube.exe" tunnel
* Tunnel successfully started

* NOTE: Please do not close this terminal as this process must stay alive for the tunnel to be accessible ...

* Starting tunnel for service heart-disease-api.
E1019 23:28:54.584139    1788 ssh_tunnel.go:148] error starting ssh tunnel: exec: "ssh": executable file not found in %PATH%
* Stopped tunnel for service heart-disease-api.
PS C:\Users\91620\MLOps-Assignment> kubectl port-forward service/heart-disease-api 8001:8000
Forwarding from 127.0.0.1:8001 -> 8000
Forwarding from [::1]:8001 -> 8000
Handling connection for 8001
Handling connection for 8001
Handling connection for 8001
Handling connection for 8001
```

10. Monitoring, Logging, and Reproducibility

Basic logging was checked through the Uvicorn output and Kubernetes logs. The command kubectl logs deployment/heart-disease-api --tail=50 displays recent logs from the application container. These logs help confirm application startup and show requests reaching the service. The health endpoint provides a simple operational check for service availability and model loading. This is a basic monitoring setup rather than a full monitoring platform with dashboards, alerts, or data-drift detection.

Several project artifacts support reproducibility: requirements.txt records dependency versions; the saved pipeline preserves the fitted preprocessing and estimator; reports/data_split_indices.json records split information; notebooks and scripts document the workflow; mlflow.db and mlruns preserve local experiment tracking; and the GitHub Actions workflow automates configured checks. Reproducibility still depends on having the required data, artifacts, and compatible environment available when the project is rerun.

- Log command: kubectl logs deployment/heart-disease-api --tail=50

```
PS C:\Users\91620\MLOps-Assignment> kubectl logs deployment/heart-disease-api --tail=50
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:34462 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:34462 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:34464 - "GET /health?utm_source=chatgpt.com HTTP/1.1" 200 OK
INFO:     127.0.0.1:51236 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:51236 - "GET /openapi.json HTTP/1.1" 200 OK
PS C:\Users\91620\MLOps-Assignment> kubectl get deployment,service,pods
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api   1/1     1            1           11m

NAME                         TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/heart-disease-api    LoadBalancer   10.98.128.229   <pending>     8000:31614/TCP   11m
service/kubernetes           ClusterIP      10.96.0.1       <none>        443/TCP          16m

NAME                                    READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-6db5cf645c-2wmtx   1/1     Running   0          11m
PS C:\Users\91620\MLOps-Assignment> kubectl port-forward service/heart-disease-api 8001:8000
Forwarding from 127.0.0.1:8001 -> 8000
Forwarding from [::1]:8001 -> 8000
Handling connection for 8001
Handling connection for 8001
Handling connection for 8001
Handling connection for 8001
Handling connection for 8001
Handling connection for 8001
```

11. Conclusion, Limitations, and Submission Checklist

Through this assignment, I implemented and documented a local MLOps workflow that connects data preparation, EDA, feature processing, model comparison, experiment tracking, API development, automated testing, Docker packaging, and Kubernetes deployment. The API health check and local prediction service were verified. This helped me understand that an MLOps project includes the engineering needed to serve and verify a model, not just the training code.

The main limitations are that Kubernetes access is local-only through port forwarding, the Minikube tunnel could not start because SSH was missing from PATH, and two MLflow runs failed. Metric files should be reconciled carefully so each reported value is tied to the correct model and evaluation split. The dataset is small, and its results should not be generalized to real-world clinical use. The classifier is an educational demonstration and not a medical diagnostic tool.

The repository contains the report, notebooks and scripts, API source code, saved model, deployment manifests, screenshots, dependency list, and CI workflow.

- Repository: 2025ct05044-cmd: MLOps-Assignment
- Submission Link: submission

Contains Report document